HOSTEL MANAGEMENT SYSTEM

UNIT TESTING REPORT

Hostel Management System

Quaid-i-Azam University

Software Testing Documentation

Document status	Prepared test plan and test cases; execution results pending
Version / date	1.0 | 26 October 2026
System context	Hostel Management System (HMS), Quaid-i-Azam University
Report basis	Project workflows taken from supplied performance-testing report; source code and implementation details not supplied

Hostel Management System - Quaid-i-Azam University

Document Control

Field	Details
Document title	Unit Testing Report
Project	Hostel Management System
Institution	Quaid-i-Azam University
Document version	1.0
Prepared for	Project testing and report documentation
Testing status	Proposed unit test cases documented; no execution evidence supplied
Repository destination	reports/unit-testing-report.docx on the report branch

1. Introduction

This report defines a unit-testing approach for the Hostel Management System (HMS) described in the supplied project report. Unit testing verifies small, isolated functions, methods, validators, and service logic using controlled inputs and mocked dependencies. The source code, framework, and test results were not provided, so cases are expressed as behavior-focused targets to map to actual modules and are not evidence of tests already executed.

2. Objectives

• Verify core business rules independently of the user interface and live database.

• Cover valid, invalid, boundary, and exceptional inputs for critical calculations and state transitions.

• Use mocks or fixtures to isolate units from external services and prevent writes to production data.

• Record test framework, coverage, execution results, and defects so changes can be regression-tested.

3. Scope

Unit / module	Unit-level behavior to verify
Authentication helpers	Credential validation, password policy helpers, role checks, and session/token utility behavior.
Application rules	Required-field validation, eligibility rules, and allowed application status transitions.
Room / bed allocation	Availability and eligibility rules, conflict detection, capacity/occupancy calculations.
Complaint logic	Required data checks and allowed complaint status transitions.
Fee logic	Fee amount/status validation and supported verification state transitions.
Visitor logic	Visitor field validation and entry/exit state rules.
Notice logic	Required content, audience/visibility rules, and publication state transitions.
Report calculations	Occupancy totals and filtering/aggregation logic on controlled records.
Shared utilities	Date handling, identifiers, input normalization, and error mapping used by HMS modules.

Out of scope: integrated workflows, browser/UI behavior, database integration, load testing, penetration testing, and production acceptance. Those require separate test levels.

4. Test Environment and Preconditions

Environment item	Value to record before testing
Repository / branch / commit	_____
Language and framework versions	_____
Unit test runner and version	_____
Coverage tool and version	_____
Mocking / fixture strategy	_____
Test configuration	_____
Test date and tester	_____

Preconditions

• Review the codebase and map each behavior below to the actual function/class under test.

• Use isolated test configuration, deterministic fixtures, and mocks for database, email, payment, and other external services.

• Ensure tests do not connect to production services or use real student, visitor, or payment data.

• Define expected behavior and boundary rules with the project team where requirements are not documented.

• Run tests with a clean database-independent setup where possible and retain test output and coverage report.

5. Unit Test Cases

These are proposed behavior targets, not assertions about the implementation. Adapt names and expected rules after reviewing the source code and requirements. Status is Not run until execution evidence is recorded.

ID	Unit / behavior	Inputs / expected result	Status
UT-01	Required-field validator	Valid complete data passes; missing required fields fail with field-specific validation results.	Not run
UT-02	Email / identifier validation	Valid formats are accepted and malformed or unsupported values are rejected consistently.	Not run
UT-03	Password policy helper	Passwords at and beyond documented length/complexity limits produce the expected result without exposing the secret.	Not run
UT-04	Role permission helper	Allowed role/action combinations pass; disallowed and unknown roles are denied by default.	Not run
UT-05	Application eligibility rule	Eligible test profile meets configured rules; missing or disqualifying conditions return the expected reason.	Not run
UT-06	Application status transition	Only documented transitions are accepted; invalid transitions leave the original state unchanged.	Not run
UT-07	Application duplicate check	Existing duplicate criteria are detected; distinct applications remain valid according to requirements.	Not run
UT-08	Bed capacity calculation	Available/occupied counts for empty, partial, full, and boundary-capacity fixtures match expected results.	Not run
UT-09	Bed allocation conflict rule	Available bed for eligible student passes; occupied bed or conflicting active allocation is rejected.	Not run
UT-10	Allotment state transition	Valid assign/change/cancel transitions update expected state; invalid transition is rejected without side effects.	Not run
UT-11	Complaint validation and status	Required complaint fields and allowed status transitions are enforced; unsupported state changes fail.	Not run
UT-12	Fee amount / status rule	Valid nonnegative amounts and permitted fee state changes pass; invalid amounts or transitions fail.	Not run
UT-13	Visitor entry/exit rule	Entry creates expected active visit state; exit closes it; duplicate or out-of-order transitions are handled as specified.	Not run
UT-14	Notice visibility rule	Published notices are returned for intended audience; draft or restricted notice is excluded for other audiences.	Not run
UT-15	Occupancy aggregation	Controlled room/bed/allotment fixtures produce expected totals, including empty and fully occupied cases.	Not run
UT-16	Date boundary handling	Dates at configured check-in/out, deadline, or reporting boundaries are classified according to documented timezone rules.	Not run
UT-17	Error mapping helper	Known validation/domain errors map to safe expected results; unexpected exceptions do not expose internals.	Not run
UT-18	External dependency isolation	Mocked database/payment/email dependency is called with expected arguments; failures are propagated or handled as specified.	Not run

6. Coverage and Acceptance Criteria

Agree numeric coverage targets with the project team; coverage percentage alone does not demonstrate correctness. Report line/branch coverage for critical business-rule modules where the tool supports it.

- All agreed unit cases pass in the documented test environment.
- Critical business rules include valid, invalid, and boundary-condition tests.
- External dependencies are mocked or isolated; unit tests are deterministic and do not require production services.
- No unresolved critical unit-test failures remain; accepted exceptions are documented with an owner.
- Coverage report and execution output are attached to the test record, with changed modules and exclusions identified.

7. Results and Measurements

No source code, unit test output, coverage report, or test runner information was supplied. This section remains pending and does not imply a pass or fail.

Module / case group	Passed	Failed	Not run	Coverage / evidence
Validation and access / UT-01 to UT-04	___	___	___	_____
Applications and allocation / UT-05 to UT-10	___	___	___	_____
Services and shared rules / UT-11 to UT-18	___	___	___	_____

7.1 Defect and Observation Log

ID	Finding / evidence	Severity	Owner	Status
UNIT-01	To be completed after execution	TBD	TBD	Open
UNIT-02	Add a row for each distinct finding	TBD	TBD	Open

8. Execution Notes and Report Limits

This is a proposed unit test plan derived from the feature areas listed in the supplied performance-testing report. Function names, code structure, business rules, coverage targets, and actual outcomes must be confirmed against the HMS source code and project requirements before sign-off.

9. Sign-off

Role	Name	Signature / date
Tester / developer	_____	_____
Project supervisor	_____	_____
System owner / approver	_____	_____